

Practice - Ex2

Build a modular REST API for a news platform using the **MVC pattern** — separating concerns into models, controllers, and routes.

Learning Objectives

By completing this exercise, you will be able to:

- Structure an Express application using the MVC (Model-View-Controller) pattern
 - Separate data, business logic, and routing into dedicated modules
 - Use `express.Router()` to create modular, mountable route handlers
 - Implement full CRUD operations across multiple related resources
 - Import and export modules using ES Module syntax
-

Getting Started

1. Navigate to the `EX-2/` folder
2. Run `npm install` to install dependencies
3. Examine the existing project structure:

```
EX-2/
├─ server.js           ← Entry point (provided)
├─ models/
│   └─ data.js         ← In-memory data store (provided)
├─ controllers/        ← Your controller logic goes here
│   └─ (empty)
├─ routes/             ← Your route definitions go here
│   └─ (empty)
└─ package.json
```

1. Open `models/data.js` and study the data — you'll be building APIs around these 3 entities:
 - **Articles** — has `id`, `title`, `content`, `journalistId`, `categoryId`
 - **Journalists** — has `id`, `name`, `email`
 - **Categories** — has `id`, `name`
-

Tasks

Complete the following tasks **in order**. Each task builds on the previous one.

Task 1 — Understand the Data Model

- Open `models/data.js` and study the three arrays: `articles`, `journalists`, `categories`
 - Note the relationships: each article references a `journalistId` and a `categoryId`
 - Understand what fields are available for each entity — you will need these for your CRUD operations
-

Task 2 — Create Article Controller

- Create a new file: `controllers/articleController.js`
 - Import the `articles` array from `../models/data.js`
 - Implement 5 controller functions (one for each CRUD operation):
 - Get all articles
 - Get a single article by ID
 - Create a new article (validate required fields: `title`, `content`, `journalistId`, `categoryId`)
 - Update an article by ID (update only provided fields)
 - Delete an article by ID
 - Export all controller functions
-

Task 3 — Create Article Routes

- Create a new file: `routes/articleRoutes.js`
 - Import `express` and create a `Router` instance
 - Import the controller functions from `../controllers/articleController.js`
 - Define routes that map HTTP methods to controller functions
 - Export the router
-

Task 4 — Mount Article Routes in Server

- Open `server.js`

- Add the JSON body parser middleware
- Import the article router
- Mount it at the `/articles` path using `app.use()`
- Test all 5 article endpoints before moving on

Checkpoint: All article CRUD operations should work before proceeding.

Task 5 — Create Journalist Controller

- Create a new file: `controllers/journalistController.js`
 - Import the `journalists` array from `../models/data.js`
 - Implement 5 controller functions (same CRUD pattern as articles)
 - Validate required fields for creation: `name`, `email`
 - Export all controller functions
-

Task 6 — Create Journalist Routes

- Create a new file: `routes/journalistRoutes.js`
 - Follow the same pattern as article routes
 - Map HTTP methods to journalist controller functions
 - Export the router
-

Task 7 — Mount Journalist Routes in Server

- Import the journalist router in `server.js`
 - Mount it at the `/journalists` path
 - Test all 5 journalist endpoints
-

Task 8 — Create Category Controller

- Create a new file: `controllers/categoryController.js`
- Import the `categories` array from `../models/data.js`
- Implement 5 controller functions (same CRUD pattern)
 - Validate required fields for creation: `name`

- Export all controller functions

Task 9 — Create Category Routes

- Create a new file: `routes/categoryRoutes.js`
- Follow the same pattern as previous routes
- Map HTTP methods to category controller functions
- Export the router

Task 10 — Mount Category Routes in Server

- Import the category router in `server.js`
- Mount it at the `/categories` path
- Test all 5 category endpoints

API Endpoint Reference

Articles

Method	Endpoint	Description	Success Status	Error Status
GET	<code>/articles</code>	List all articles	200	—
GET	<code>/articles/:id</code>	Get a single article	200	404 Not Found
POST	<code>/articles</code>	Create a new article	201	400 Bad Request
PUT	<code>/articles/:id</code>	Update an article	200	404 Not Found
DELETE	<code>/articles/:id</code>	Delete an article	204	404 Not Found

Journalists

Method	Endpoint	Description	Success Status	Error Status
GET	<code>/journalists</code>	List all journalists	200	—
GET	<code>/journalists/:id</code>	Get a single journalist	200	404 Not Found
POST	<code>/journalists</code>	Create a new journalist	201	400 Bad Request

Method	Endpoint	Description	Success Status	Error Status
PUT	/journalists/:id	Update a journalist	200	404 Not Found
DELETE	/journalists/:id	Delete a journalist	204	404 Not Found

Categories

Method	Endpoint	Description	Success Status	Error Status
GET	/categories	List all categories	200	—
GET	/categories/:id	Get a single category	200	404 Not Found
POST	/categories	Create a new category	201	400 Bad Request
PUT	/categories/:id	Update a category	200	404 Not Found
DELETE	/categories/:id	Delete a category	204	404 Not Found

Expected Final Structure

When you're done, your project should look like this:

```
EX-2/
├─ server.js
├─ models/
│   └─ data.js
├─ controllers/
│   ├── articleController.js
│   ├── journalistController.js
│   └─ categoryController.js
├─ routes/
│   ├── articleRoutes.js
│   ├── journalistRoutes.js
│   └─ categoryRoutes.js
└─ package.json
```

Testing Your API

Start your server:

```
node server.js
```

Using curl

```
# --- Articles ---
# GET all articles
curl <http://localhost:3000/articles>

# GET article by ID
curl <http://localhost:3000/articles/1>

# CREATE an article
curl -X POST <http://localhost:3000/articles> \
  -H "Content-Type: application/json" \
  -d '{"title": "Breaking News", "content": "Something happen
ed...", "journalistId": 1, "categoryId": 2}'

# UPDATE an article
curl -X PUT <http://localhost:3000/articles/1> \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Title"}'

# DELETE an article
curl -X DELETE <http://localhost:3000/articles/1>

# --- Journalists ---
# GET all journalists
curl <http://localhost:3000/journalists>

# CREATE a journalist
curl -X POST <http://localhost:3000/journalists> \
  -H "Content-Type: application/json" \
  -d '{"name": "New Reporter", "email": "reporter@newsco.co
m"}'

# --- Categories ---
# GET all categories
curl <http://localhost:3000/categories>
```

```
# CREATE a category
curl -X POST <http://localhost:3000/categories> \\  
  -H "Content-Type: application/json" \\  
  -d '{"name": "Science"}'
```

Using Thunder Client / Postman

1. Open Thunder Client (or Postman)
 2. Create requests matching the endpoint tables above
 3. For POST and PUT requests, set the body to JSON format
 4. Verify the response status codes match the expected values
 5. Test each resource (articles, journalists, categories) independently
-

Submission

Submit your completed work using **one** of the following methods:

- **GitHub:** Push your code to a repository and share the link
 - **Zip:** Compress the `EX-2/` folder (exclude `node_modules/`) and submit the `.zip` file
-

Good luck! 🚀